

Especificação Técnica e Arquitetural Completa do Sistema (PDSI-I)

Este documento representa o manual técnico exaustivo e a especificação de arquitetura de software para o projeto da disciplina de Desenvolvimento de Sistemas para Internet I. O objetivo deste material não é apenas apresentar um roteiro de requisitos, mas sim detalhar as motivações, as decisões de engenharia, a infraestrutura e a implementação de cada aspecto da aplicação, desde o planejamento visual até as camadas de segurança e persistência de estado.

1. Visão Geral do Sistema e Arquitetura de Alto Nível

A internet evoluiu de páginas estáticas e vinculadas por hiperlinks para aplicações robustas, que simulam o comportamento e a performance de aplicativos nativos de desktop ou mobile. Este projeto baseia-se nesta evolução, adotando o modelo arquitetural conhecido como **Aplicações de Página Única (Single Page Applications - SPA)** acoplado a uma arquitetura orientada a serviços (API RESTful).

1.1. Motivação e O Problema de Negócio

O sistema foi concebido para resolver necessidades transacionais de usuários em tempo real. Diferente das antigas aplicações web onde cada clique resultava em um recarregamento total da tela ("tela branca" e novo download de todo o HTML/CSS), a arquitetura adotada aqui prevê o carregamento de uma única "casca" de interface gráfica (shell) no primeiro acesso. Todo o tráfego subsequente restringe-se exclusivamente aos dados brutos em formato JSON, otimizando drasticamente a banda de rede e a velocidade percebida pelo usuário.

1.2. Separação de Responsabilidades (Frontend vs Backend)

A aplicação adota um modelo "Headless" estrito.

- **O Frontend (O foco deste projeto):** É um aplicativo independente que roda inteiramente no navegador web (cliente). Ele é responsável por renderizar o DOM, lidar com os eventos de mouse e teclado, validar regras primárias de formulários e manter o estado visual da interface.
 - **O Backend (A API):** Fica em um servidor remoto, desconhecendo completamente o HTML ou a interface gráfica. Sua única função é receber requisições do Frontend, checar as regras de negócio complexas, interagir com o Banco de Dados e devolver respostas através de códigos HTTP (200 OK, 404 Not Found, 401 Unauthorized, etc.) e corpos em formato JSON.
-

2. Metodologias, Estruturação de Código e versionamento

Para que um sistema grande possa ser sustentado e expandido sem se tornar um emaranhado de código ilegível, adotam-se rígidos padrões de estruturação.

2.1. Versionamento Distribuído (Git e Git Flow)

O código não é guardado em arquivos ".zip" ou pastas datadas. Usa-se o **Git**. A estratégia de ramificação (branching) adotada é essencial:

- A branch **main** ou **master** espelha o código exato que está rodando em produção (o que o usuário acessa). É proibido programar diretamente nela.
- A branch **develop** contém as funcionalidades da próxima versão.
- As branches de **feature/nome-da-funcionalidade** são criadas a partir da **develop**. Cada programador desenvolve sua parte de forma isolada e submete um "Pull Request" para revisão por pares antes do código ser mesclado.

2.2. Arquitetura de Pastas Profunda

Dentro do ecossistema Frontend, os diretórios são organizados pela função técnica de cada arquivo:

- **/public**: Contém os assets estáticos de entrada, como o **index.html** raiz, o **favicon** e o **manifest.json** (caso a aplicação adote características de PWA - Progressive Web App).
- **/src**: O coração do projeto. Onde todo o código não compilado reside.
 - **/src/assets**: Mídias locais, fontes hospedadas e ícones em SVG otimizados.
 - **/src/components**: A base do reaproveitamento. Botões, inputs textuais, modais, alertas de erro. Eles são agnósticos ao contexto, ou seja, um componente "Botão Primário" não sabe em qual página ele está sendo usado, ele apenas recebe propriedades (props) e emite eventos de clique.
 - **/src/pages** (ou **/views**): O agrupamento de múltiplos componentes que formam uma tela completa (ex: Tela de Cadastro, Dashboard Principal).
 - **/src/layouts**: Estruturas envoltórias. Por exemplo, uma "Master Page" que sempre exibe o Menu Lateral e o Cabeçalho, alterando apenas o conteúdo central da página.
 - **/src/hooks** ou **/composables**: Lógicas reutilizáveis do JavaScript separadas da interface gráfica.
 - **/src/services**: Os "Embaixadores" da rede. Arquivos exclusivos que contêm as URLs do backend e executam o **fetch**. Nenhum componente visual deve fazer uma requisição de rede diretamente; eles chamam funções do diretório de serviços.
 - **/src/utils** ou **/helpers**: Funções puras de processamento (ex: formatador de moeda, calculador de CPF).

3. Fase 1: Engenharia de Interface Gráfica e Experiência do Usuário (UI/UX)

A Fase 1 é muito mais complexa do que "fazer telas bonitas". Trata-se de construir a ponte de comunicação entre a máquina e o humano.

3.1. Design System e Identidade Visual

Ao invés de definir cores e tamanhos de fonte aleatoriamente em cada arquivo CSS, a aplicação constrói um "Design System" em sua base (ex: através de Variáveis CSS ou *Design Tokens*).

- Existe uma paleta restrita: Cores primárias (para ações de confirmação), Cores semânticas (Vermelho para Erro, Verde para Sucesso, Amarelo para Atenção) e tons de cinza para texturas e bordas.

- Isso garante que, se o cliente mudar o logotipo e exigir que o botão primário deixe de ser azul e passe a ser roxo, a alteração de uma única variável global (`--color-primary: purple;`) reflita imediatamente em milhares de componentes através de toda a aplicação web.

3.2. Responsividade (Media Queries e Mobile-First)

Com mais de 60% do tráfego web originando-se de celulares, a aplicação não é "adaptada" para telas pequenas, ela nasce para elas. O layout Mobile-First força o programador a decidir o que é crucial na tela, empilhando os componentes verticalmente. Conforme a tela cresce (breakpoints identificados por media queries no CSS, como `min-width: 768px` para tablets e `min-width: 1024px` para desktops), o layout se expande utilizando a flexibilidade do **CSS Grid Layout** e **Flexbox** para aproveitar o espaço horizontal ganho, revelando menus laterais ou colunas múltiplas.

3.3. Acessibilidade Aprofundada (A11y)

O sistema deve ser navegável por indivíduos com deficiências visuais ou motoras.

- **Hierarquia de Cabeçalhos:** A página tem estritamente um `<h1>` por vez. Os `<h2>` e `<h3>` seguem uma ordem lógica sem pular níveis, para que leitores de tela criem um índice mental correto da página.
- **Foco Visível:** O uso da tecla TAB pelo teclado evidencia visualmente qual elemento interativo (link, botão, input) está focado, através de uma propriedade `outline` de alto contraste.
- **ARIA (Accessible Rich Internet Applications):** Para componentes não nativos. Exemplo: um div que funciona como botão terá o atributo `role="button"` e `tabindex="0"`, e modais escondidos terão `aria-hidden="true"` para que o leitor de tela não anuncie elementos invisíveis.

4. Requisitos Obrigatórios da Fase 2 (Detalhados Exaustivamente)

Nesta seção, isolamos e esmiuçamos cada um dos requisitos obrigatórios que conferem ao projeto seu caráter de "Aplicação de Internet".

Requisito A: Comunicação Síncrona e Assíncrona (Fetch API)

O Paradigma: A internet primitiva era puramente síncrona: o navegador solicitava uma URL, ficava travado esperando a rede, recebia o HTML, limpava a tela atual e renderizava a nova. O JavaScript moderno introduziu o conceito de assincronicidade (Event Loop).

A Implementação com Fetch:

- A **Fetch API** é o método padrão dos navegadores para orquestrar comunicações de rede (substituindo o antigo e verboso `XMLHttpRequest`).
- **Fluxo Assíncrono:** Quando a aplicação precisa submeter o perfil atualizado de um usuário, ela chama o `fetch('https://api.projeto.com/users/1', { method: 'PUT', body: JSON.stringify(dados) })`. Essa função devolve uma **Promise** (Promessa).
- **A Promessa e a Thread de UI:** Ao invés de paralisar a renderização do navegador esperando a rede (o que poderia levar longos segundos em conexões 3G), o JavaScript instrui o motor de rede do navegador a fazer o trabalho em segundo plano. O código JS avança, permitindo que animações e interações continuem na tela. Quando a rede responde (seja com sucesso ou com timeout), a Promise é "resolvida" e o JavaScript executa o bloco `then()` ou a linha subsequente a um `await`.

- **Tratamento de Exceções de Rede:** O `fetch` tem uma peculiaridade arquitetural importante. Ele só rejeita a Promessa se houver uma falha catastrófica de infraestrutura (ex: o DNS não encontrou o domínio ou a internet do cliente caiu). Se o servidor backend retornar um Erro 500 (Internal Server Error) ou 404 (Not Found), a Promessa **não falha automaticamente**. O sistema deve inspecionar manualmente a propriedade booleana `response.ok`. Se for falsa, a aplicação dispara manualmente um erro (`throw new Error(...)`), que é capturado em um bloco `catch` para exibir um Toast (mensagem flutuante) vermelha na interface do usuário.

Requisito B: Web Storage API (Local e Session)

O Paradigma: Requisições de rede são caras (gastam bateria do celular do usuário, banda de dados e tempo de processamento no servidor backend). O navegador provê pequenos bancos de dados nativos para aliviar essa carga.

A Implementação no Projeto: A Web Storage API é síncrona. Ou seja, ao chamar `localStorage.getItem()`, o JavaScript paralisa por uma fração de milissegundos para ler o disco do usuário. Portanto, armazena-se estritamente dados leves (em torno de 5MB limite). Todos os dados ali são strings puras.

- **LocalStorage:** Os dados sobrevivem mesmo que o computador seja reiniciado.
 - *Aplicação Profunda no Sistema:* Excelente para retenção de configurações de UI (User Interface). Se o usuário clicou no botão "Modo Escuro", gravamos `localStorage.setItem('theme', 'dark')`. Toda vez que o sistema iniciar, antes mesmo do primeiro frame ser renderizado, o JavaScript lê essa chave e aplica a cor escura, evitando aquele "piscar branco" de interface. Também pode ser usado como cache de listas imutáveis (ex: lista de estados brasileiros em um select).
- **SessionStorage:** A efemeridade como recurso de proteção de fluxo.
 - *Aplicação Profunda no Sistema:* Os dados morrem instantaneamente se a aba (tab) específica for fechada. É utilizado em fluxos de negócios sensíveis ao contexto da navegação atual. Por exemplo, em um processo de cadastro com três telas sucessivas (Passo 1, Passo 2, Passo 3). Os dados do Passo 1 são salvos no `sessionStorage`. Se o usuário clicar no botão de "Voltar" do navegador, a aplicação lê o session e preenche os campos do Passo 1 novamente, garantindo uma UX incrível. Mas se ele fechar a aba de raiva, o rascunho some.

Requisito C: HTTP Cookies

O Paradigma: Diferente do Web Storage, cujo uso depende que o desenvolvedor Front-end escreva um código JavaScript mandando ler ou gravar, o Cookie é uma tecnologia orgânica da rede. O navegador o envia silenciosamente e automaticamente em **todas** as requisições para o domínio proprietário dele.

A Implementação no Projeto:

- Devido a essa automação de envio na rede, Cookies pesados degradariam a performance de upload de cada clique do usuário. Portanto, seu tamanho é minúsculo (geralmente sob 4KB) e seu conteúdo restrito a **IDs de rastreamento** (como IDs do Google Analytics para entender o fluxo de telas que o usuário percorre) ou, crucialmente, para manutenção de sessão na arquitetura de autenticação (veja o próximo requisito).
- A leitura via JavaScript `document.cookie` é verbosa, logo abstrações ou pequenos utilitários são frequentemente escritos para extrair o valor da chave específica (através de métodos de

manipulação de strings ou Regex).

Requisito D: Autenticação (Métodos e Fluxos)

O Paradigma: A web (protocolo HTTP) é *Stateless* (não tem estado). O servidor não lembra quem você é a cada nova requisição. Se a página A pede dados e a página B pede dados 2 segundos depois, para o servidor, parecem pessoas completamente diferentes. A Autenticação é o processo de atar a identidade a cada requisição.

A Implementação e a Decisão Técnica: O sistema pode adotar uma dentre as metodologias de ponta, sendo o uso de **JWT (JSON Web Tokens)** a abordagem arquitetural mais compatível com Frontends desacoplados (SPAs).

- **O Fluxo Detalhado com JWT:**

1. O usuário envia o form de login usando **Fetch API** em POST, submetendo `{ "email": "user@a.com", "senha": "123" }`.
2. O servidor processa e retorna na resposta JSON uma string imensa com três partes separadas por pontos (**cabecalho.payload.assinatura**), o tal JWT. Ele contém os papéis (Roles) do usuário e o seu tempo de expiração.
3. A segurança da arquitetura entra aqui: A aplicação precisa guardar esse token. Se guardar no **localStorage**, ele será vulnerável a ataques XSS. A abordagem "State of the Art" (Estado da Arte) é o backend devolver esse token diretamente em um cabeçalho HTTP **Set-Cookie** com a flag **HttpOnly**. Isso significa que o Token fica dentro do navegador do usuário, mas o JavaScript local é **bloqueado e incapaz** de ler esse Cookie.
4. Sempre que a aplicação usar a **Fetch API** para buscar dados protegidos na Fase 2, o navegador anexa magicamente o Cookie **HttpOnly** com o token. O servidor o lê, valida a criptografia, e libera o acesso. O usuário está autenticado e o token está blindado contra injeções de script no frontend.

- **Alternativa: WebAuthn:** Caso o projeto queira extrapolar, implementa-se a **Web Authentication API**, onde o JavaScript aciona o chip TPM da placa-mãe (Windows Hello, Touch ID do Mac) para criar uma credencial de chave criptográfica assimétrica. Não há envio de senhas na rede. Há apenas assinaturas de desafios do servidor através da chave privada armazenada a sete chaves no hardware do usuário.

Requisito E: Mecanismos de Segurança (Além da Autenticação)

O Paradigma: Um cofre com uma porta trancada não protege o dinheiro se a janela estiver aberta. A autenticação garante quem é o usuário, mas outros mecanismos defendem a infraestrutura de fraudes e manipulação de terceiros.

A Implementação Defensiva no Frontend:

1. **Proteção contra Cross-Site Scripting (XSS):** Um ataque XSS ocorre quando a API retorna um texto de um banco de dados que contém, na verdade, um código JS: `<script>enviarTokenParaHacker()</script>`. Se a aplicação inserir isso num painel usando manipulação grosseira do DOM (`elemento.innerHTML = texto`), o script roda. A arquitetura exige que todo dado dinâmico inserido no DOM passe por rotinas de "escape" (transformando `<` em `<`), geralmente automatizadas por frameworks ou utilizando APIs seguras como o `document.createTextNode()`.

2. **Proteção contra Cross-Site Request Forgery (CSRF):** Impede que um site falso (www.gatinhosfofos.com) execute scripts que acionam a API do seu sistema (www.projetoPDSI.com) aproveitando-se do fato de que o usuário tem um cookie de login ativo. Para mitigar isso no projeto, todos os cookies de sessão são marcados com o atributo `SameSite=Strict` ou `Lax`. Assim, o navegador se recusa a enviar o Cookie de autenticação se a requisição originou-se de outro domínio que não seja o do Frontend original.
3. **Configuração de CORS (Cross-Origin Resource Sharing):** Embora seja implementado no servidor Backend, ele é exigido pelo navegador do Frontend. O servidor dita os domínios restritos que podem invocar a API (enviando os headers `Access-Control-Allow-Origin`). Se o Frontend do projeto não for listado ali, o próprio navegador cliente da vítima bloqueia a leitura da resposta, frustrando ataques laterais.

Requisito F: Validação de Formulários Avançada

O Paradigma: Enviar dados corrompidos, vazios ou mal formatados para o banco de dados degrada o sistema e consome poder computacional do servidor de forma estúpida. O Frontend é a primeira linha de defesa.

A Implementação (Estratégia de Duas Camadas): A validação neste projeto é construída com uma UX proativa e reativa.

- **A Abordagem Nativa (Constraint Validation API):** O primeiro nível de validação usa as ferramentas nativas do HTML5 sem necessitar de JavaScript manual para tudo.
 - Campos numéricos recebem `type="number" min="0" max="100"`. E-mails recebem `type="email"`. Textos recebem máscaras via atributos `pattern="[0-9]{3}\.[0-9]{3}\.[0-9]{3}-[0-9]{2}"` (para um CPF, por exemplo).
 - Se o usuário clica no botão Submit violando isso, o próprio navegador aborta o envio e projeta balões de dica apontando o erro, sem gastar sequer 1 linha de JS.
- **A Abordagem Assíncrona e Lógica via JavaScript:** O HTML5 não consegue validar regras cruzadas (Exemplo: A "Data Final" deve ser obrigatoriamente superior à "Data Inicial", ou as senhas nos campos "Senha" e "Confirmar Senha" devem ser um clone perfeito).
 - O evento `submit` do formulário é interceptado via `.addEventListener('submit', function(e) { ... })` e interrompido preventivamente (`e.preventDefault()`).
 - A lógica analisa os valores do DOM, testa expressões regulares (Regex) avançadas para exigir senhas fortes (no mínimo 8 letras, 1 símbolo e 1 caixa alta).
 - Se um erro é detectado, a classe CSS daquele input específico muda (sua borda fica vermelha) e uma tag `<small>` abaixo dele é preenchida dinamicamente no DOM com a mensagem explicativa. Isso confere ao sistema o caráter de uma aplicação profissional.
 - **A "Dupla Garantia":** O Frontend valida por UX (Experiência do Usuário). Contudo, a arquitetura assume a premissa de que *o cliente nunca é confiável* (um programador experiente poderia desabilitar o JavaScript pelo DevTools). Portanto, o Backend obrigatoriamente re-valida os dados. Quando a `Fetch API` envia o POST e recebe um HTTP 422 Unprocessable Entity, o Frontend captura o JSON de erro do backend e injeta essas mensagens na interface.

5. Estratégia de Build, Bundle e Implantação (Deploy)

Por fim, o código humano espalhado em centenas de pastas e arquivos `.js`, `.css` ou frameworks precisa ser convertido para algo que os navegadores carreguem de forma ultra eficiente na internet.

1. **Bundling (O Empacotamento):** Ferramentas como Vite ou Webpack varrem toda a árvore de dependências do projeto (do `index.js` em diante) e as unificam. Elas executam "Minificação" (remoção de comentários, espaços em branco e renomeação de variáveis gigantes para letras simples) para reduzir drasticamente o tamanho do arquivo em Kb.
2. **Tree-Shaking:** Funções ou arquivos exportados que nunca foram utilizados em nenhuma parte do código são cirurgicamente removidos do pacote final para economizar banda.
3. **Ambiente de Produção (Deploy):** Como o resultado de um sistema Frontend desacoplado é pura e simplesmente arquivos estáticos unificados (`index.html`, alguns pacotes de `.js` ofuscado e `.css`), a implantação não exige servidores virtuais complexos ou instâncias em nuvem rodando processos pesados (como Apache ou Nginx lidando com PHP/Python). Os arquivos são enviados para Redes de Distribuição de Conteúdo (CDNs, como AWS CloudFront, Vercel ou Netlify). Eles são armazenados em *Edge Servers* localizados fisicamente próximos aos usuários no globo inteiro. Quando um usuário do Brasil digita a URL da aplicação, os arquivos são entregues a partir de um servidor em São Paulo em poucos milissegundos, garantindo uma abertura de tela fulminante e imune a quedas por pico de acesso.

Conclusão da Especificação

Este sistema se materializa não apenas como um projeto acadêmico de preenchimento de requisitos, mas sim como uma **Plataforma de Engenharia Web completa**. Cada decisão técnica tomada - desde a abstração visual com *Design Systems*, a passagem para um tráfego de dados assíncrono blindado contra XSS e a defesa multicamada de formulários - corrobora para a construção de um software que atende a critérios profissionais, comerciais e de altíssima resiliência e usabilidade.